

SmartTask

سامانه مدیریت هوشمند وظایف چابک

نوع پروژه: پایان نامه کارشناسی

تکنولوژی اصلی: ASP.NET Core 8.0 MVC | SQL Server | SignalR | AI (Qwen3-4B)

روش شناسی: Agile / Scrum

فهرست مطالب

۱. نمای کلی پروژه | ۲. معماری و زیرساخت فنی | ۳. لایه‌بندی معماری | ۴. موجودیت‌های داده‌ای | ۵. فیچرهای پایه‌ای | ۶. یازده نوآوری اصلی | ۷. تکنولوژی‌ها و چارایی استفاده | ۸. فایل‌ساختار | ۹. تست‌ها | ۱۰. پرسش‌های متداول

۱. نمای کلی پروژه

SmartTask چیست؟

SmartTask یک سامانه وب‌محور مدیریت وظایف چابک (Agile Task Management) است که بر اساس روش‌شناسی Scrum طراحی شده. هدف اصلی پروژه این است که فراتر از ابزارهای ساده مدیریت تسک، ابزارهای هوشمند و خودکاری در اختیار مدیران پروژه و اعضای تیم قرار دهد.

اهداف کلیدی

- تحلیل خودکار بار کاری اعضا بر اساس ساعت تخمینی و ظرفیت هفتگی
- شناسایی وابستگی بین وظایف و محاسبه زنجیره تأثیر تأخیرها
- اولویت‌بندی هوشمند با الگوریتم امتیازدهی (نه فقط حس مدیر)
- پیش‌بینی ریسک تأخیر و نمایش بصری وضعیت پروژه
- گزارش‌گیری AI در پایان هر اسپرینت
- تجزیه وظایف با AI برای کمک به اعضای کم‌تجربه
- مبادله تسک بین اعضا با سیستم درخواست و تأییدیه
- ثبت کارهای فوری خارج از دامنه (آفرود) بدون آسیب به ساختار رسمی

مخاطبان سامانه

نقش	دسترسی
Admin	مدیریت کل سامانه، کاربران، داشبورد ادمین
ProjectManager	مدیریت پروژه‌ها، اسپرینت‌ها، تیم‌ها، مشاهده تحلیل‌ها
Member	ثبت وظایف، ترید تسک، مشاهده داشبورد

ساختار سازمانی سامانه

Workspace (فضای کاری)

└─ Project (پروژه)

└─ Backlog (بکلاگ)

| └─ UserStory (داستان کاربری)

| └─ TaskItem (وظیفه اجرایی)

| └─ SubTaskItem (زیروظیفه)

| └─ TaskAssignment (تخصیص به عضو)

| └─ TaskDependency (وابستگی)

| └─ Comment, Attachment, Checklist, Label, TimeLog

└─ Sprint (اسپرینت)

└─ OffroadTask (کارهای آفرود)

└─ TaskTradeRequest (درخواست ترید)

└─ ProjectMember (اعضای پروژه)

۲. معماری و زیرساخت فنی

تکنولوژی‌های استفاده‌شده

لایه	تکنولوژی	نسخه
Framework	ASP.NET Core MVC	8.0
ORM	Entity Framework Core	8.0.28
Authentication	ASP.NET Core Identity	8.0.8
Real-time	SignalR	built-in
Database	Microsoft SQL Server	-
PDF	QuestPDF	2026.7.2
Excel	ClosedXML	0.105.1
AI	OpenAI-compatible API (LM Studio + Qwen3-4B)	-
Email	System.Net.Mail (SMTP)	-
Push Notification	Webpushr	API v1
Testing	xUnit + Moq + EF InMemory	-

چرا ASP.NET Core 8.0؟

- عملکرد بالا: Kestrel web server بهینه‌شده
- **Dependency Injection** داخلی و قدرتمند
- **SignalR** برای ارتباط بلادرنگ (Real-time)
- **Entity Framework Core** برای ORM قدرتمند
- پشتیبانی عالی از **Identity** برای احراز هویت و نقش‌ها

چرا SignalR؟

SignalR چیست؟ SignalR یک کتابخانه میکروسافت برای اضافه کردن قابلیت ارتباط بلادرنگ (Real-time) به اپلیکیشن‌های وب است. به جای رفرش مداوم صفحه، سرور خودش پیام را فوراً به مرورگر می‌فرستد.

چرا در SmartTask استفاده شد؟

1. **اعلان‌های فوری:** وقتی Task جدیدی تخصیص داده شود، اعلان فوراً به کاربر می‌رسد
2. **پیام‌رسانی گروهی چت:** چت پروژه بدون رفرش صفحه کار می‌کند
3. **وضعیت آنلاین/آفلاین:** نشان دادن اینکه کی آنلاین است
4. **تایپ کردن:** نشان دادن «...در حال تایپ» به سایرین

کجا در پروژه استفاده شد؟

- `Hubs/NotificationHub.cs` — هاب اعلان‌ها
- `Hubs/ChatHub.cs` — هاب چت گروهی پروژه‌ها
- `Program.cs` — ثبت هاب‌ها: `app.MapHub<NotificationHub>("/hubs/notification")`
- `NotificationService.cs` — ارسال اعلان با `_hubContext.Clients.Group(...).SendAsync("ReceiveNotification", ...)`

۳. لایه‌بندی معماری

Controllers (MVC)	← لایه ارائه (Presentation)
Services (Implementations)	← لایه منطق تجاری (Business Logic)
Infrastructure	← Repository + UnitOfWork + BackgroundJobs
Data (DbContext + Config)	← لایه دسترسی به داده (Data Access)
Models (Entities + Enums)	← مدل‌های داده‌ای

BaseEntity — کلاس پایه تمام موجودیت‌ها

```
// SmartTask.Web/Models/Entities/BaseEntity.cs
public abstract class BaseEntity {
    public int Id { get; set; }
    public string? CreatedBy { get; set; }
    public DateTime CreatedDate { get; set; }
    public string? ChangeUser { get; set; }
    public DateTime? ChangeDate { get; set; }
    public bool ViewState { get; set; } = true; // Soft Delete
}
```

نکته مهم: فیلد `ViewState` برای **Soft Delete** استفاده می‌شود. رکوردها هرگز حذف فیزیکی نمی‌شوند و فقط `ViewState = false` می‌شوند.

Generic Repository + Unit of Work

الگوی **Repository** برای جلوگیری از تکرار کد دسترسی به دیتابیس و الگوی **UnitOfWork** برای تضمین ذخیره چند تغییر در یک تراکنش استفاده شده.

۴. موجودیت‌های داده‌ای

موجودیت	فایل	توضیح
ApplicationUser	Models/Entities/ApplicationUser.cs	کاربر با Identity، تنظیمات شخصی، ظرفیت کاری
Project	Models/Entities/Project.cs	پروژه با وضعیت، اولویت، رنگ و آیکون
Sprint	Models/Entities/Sprint.cs	اسپرینت با هدف، بازه زمانی، ظرفیت
UserStory	Models/Entities/UserStory.cs	داستان کاربری با Story Point و Business Value
TaskItem	Models/Entities/TaskItem.cs	وظیفه اجرایی با نوع، اولویت، وضعیت، ساعت تخمینی
TaskDependency	Models/Entities/TaskDependency.cs	وابستگی بین دو Task (اجباری/اختیاری)
OffroadTask	Models/Entities/OffroadTask.cs	کار آفرود خارج از دامنه رسمی
TaskTradeRequest	Models/Entities/TaskTradeRequest.cs	درخواست مبادله/واگذاری تسک
OverdueCascadeLog	Models/Entities/OverdueCascadeLog.cs	لاگ تمدید خودکار (جلوگیری از تکرار)
SprintReport	Models/Entities/SprintReport.cs	گزارش هوشمند پایان اسپرینت
Notification	Models/Entities/Notification.cs	اعلان با ۸ نوع مختلف

۵. فیچرهای پایه‌ای سامانه

احراز هویت و مدیریت نقش‌ها

فناوری: **ASP.NET Core Identity**. فایل‌ها: `AccountController.cs` , `RoleSeeder.cs` , `AdminSeeder.cs`. نقش‌ها: Admin, ProjectManager, Member. کوکی با نام SmartTask و انقضای ۷ روز.

چت گروهی پروژه

فایل‌ها: `ChatService.cs` , `ChatHub.cs`. پیام‌رسانی بلادرنگ با SignalR. قابلیت‌ها: ارسال فایل، Reply، ویرایش/حذف، Reaction ایموجی، Typing Indicator، @Mention، Pin، آنلاین/آفلاین، Rate Limiting (۵ پیام در ۳ ثانیه)، Push Notification با Webpushr.

سیستم اعلان

فایل‌ها: NotificationService.cs , NotificationHub.cs . ۸ نوع اعلان: System, Reminder, Assignment, Comment, Mention, Invitation, StatusChange, Deadline. تنظیمات شخصی اعلان‌ها. SignalR با اعلان‌های بلادرنگ.

یادآوری (Reminder)

فایل: ReminderService.cs . یادآوری دستی و خودکار. سرویس پس‌زمینه ReminderBackgroundService دوره‌ای یادآوری‌های ارسال‌نشده را بررسی می‌کند.

گزارش‌گیری

خروجی PDF با QuestPDF و Excel با ClosedXML. نمودار Burndown و Velocity.

۶. نوآوری‌های اصلی پروژه (۱۱ نوآوری)

⚠ این بخش مهم‌ترین بخش پروژه است. هر نوآوری مشکل مشخصی را حل می‌کند و SmartTask را از ابزارهای ساده متمایز می‌کند.

نواوری ۱: تجزیه هوشمند وظایف با هوش مصنوعی (AI Task Breakdown)

مشکل: وقتی تسک پیچیده ثبت می‌شود، اعضای کم‌تجربه نمی‌دانند از کجا شروع کنند.
راه‌حل: با یک کلیک، AI تسک را به ۳-۷ زیروظیفه عملی تقسیم می‌کند.

جزئیات	مقدار
سرویس	TaskBreakdownService.cs
رابط	ITaskBreakdownService.cs
کنترلر	SubTaskController.cs
مدل AI	Qwen3-4B
آدرس API	http://92.246.145.99:1234/v1/chat/completions
پرامپت	از رشته فارسی (۳ تا ۷ آیتم، هرکدام حداکثر ۸ کلمه) JSON array فقط
Temperature	۰.۵

نحوه عملکرد: کاربر دکمه «تجزیه با هوش مصنوعی» را می‌زند → عنوان و توضیحات تسک به AI ارسال می‌شود → AI JSON array برمی‌گرداند → پاسخ پارس می‌شود (Fallback به متن ساده) → لیست زیروظایف نمایش داده می‌شود.

نواوری ۲: بخش آفرود (Offroad Tasks)

مشکل: کارهای فوری خارج از Sprint (مثلاً رفع باگ سرور) اگر در Backlog وارد شوند، ساختار Sprint را بهم می‌ریزند.

راه‌حل: فضای جداگانه «آفرود» بدون آسیب به ساختار رسمی.

جزئیات	مقدار
موجودیت	OffroadTask.cs
سرویس	OffroadTaskService.cs
کنترلر	OffroadController.cs
وضعیت‌ها	ToDo, InProgress, Done, Cancelled
اولویت‌ها	Low, Normal, High, Critical

مزیت نسبت به Jira/Trello: در Jira و Trello تسک خارج از Sprint باید در Backlog وارد شود. SmartTask با بخش آفرود این مشکل را حل کرده.

نواوری ۳: تحلیل بارکاری اعضا (Workload Analysis)

مشکل: مدیر پروژه نمی‌داند هر عضو چقدر بار کاری دارد.

راه‌حل: محاسبه خودکار درصد اشغال بر اساس ساعت تخمینی و ظرفیت هفتگی.

جزئیات	مقدار
سرویس	WorkloadAnalysisService.cs
فرمول	$\text{درصد اشغال} = (\text{ساعات تخمینی یافت} \div \text{ظرفیت هفتگی}) \times 100$
وضعیت‌ها	under (80%>), balanced (80-100%), overloaded (100%<)
محدوده	تحلیل کل پروژه + تحلیل اسپرینت فعال

بهینه‌سازی: از ComputeAssignmentMap برای جلوگیری از حلقه‌های $O(n^2)$ استفاده شده.

نواوری ۴: تحلیل تأثیر وابستگی وظایف (Dependency Impact Analysis)

مشکل: وقتی تسک A عقب بیفتد، مدیر باید دستی بررسی کند کدام تسک‌ها تحت تأثیرند.

راه‌حل: پیمایش خودکار گراف وابستگی با الگوریتم BFS.

جزئیات	مقدار
سرویس	TaskDependencyService.cs
الگوریتم	BFS (جستجوی اول عرض) برای پیمایش گراف
محافظت حلقوی	WouldCreateCycleAsync() — BFS معکوس
خروجی	لیست تسک‌های تأثیرپذیر با عمق و تاریخ پیش‌بینی‌شده

مثال: اگر Task A سه روز تأخیر داشته باشد → Task B (وابسته به A) +۳ روز → Task C (وابسته به B) +۲ روز

نواوری ۵: تمدید خودکار موعد وظایف وابسته (Overdue Auto-Cascade)

مشکل: مدیر باید دستی موعد تمام تسک‌های وابسته را آپدیت کند.

راه‌حل: سرویس پس‌زمینه هر ۳۰ دقیقه تسک‌های عقب‌افتاده را شناسایی و تأخیر را خودکار منتقل می‌کند.

جزئیات	مقدار
سرویس	OverdueCascadeBackgroundService.cs
لاگ	OverdueCascadeLog.cs
فاصله اجرا	هر ۳۰ دقیقه
نوع	BackgroundService در ASP.NET Core

مکانیزم جلوگیری از تکرار: جدول OverdueCascadeLog ثبت می‌کند چند روز تأخیر اعمال شده. اگر تأخیر جدید بیشتر باشد، فقط مابه‌التفاوت اعمال می‌شود.

نواوری ۶: اولویت‌بندی هوشمند وظایف (Smart Priority System)

مشکل: اولویت‌ها معمولاً بر اساس حس مدیر تعیین می‌شوند.

راه‌حل: الگوریتم امتیازدهی با ترکیب سه عامل.

عامل	وزن	منطق
فوریت زمانی	۴۰-۰	موعد گذشته → ۴۰، امروز → ۴۰، هر روز کمتر
تأثیر وابستگی	۳۵-۰	هر تسک وابسته اجباری ۷+ (حداکثر ۲۵)
ریسک بارکاری	۲۵-۰	100% < اشغال → ۲۵، 80-100% → 15

تبدیل امتیاز: Lowest, 21-40=Low, 41-60=Medium, 61-80=High, 81-100=Highest=20-0

سرویس: PriorityEngineService.cs . قابلیت اعمال با یک کلیک.

نواوری ۷: پیش‌بینی ریسک تأخیر پروژه (Delay Risk Prediction)

مشکل: مدیر نمی‌داند پروژه چقدر در معرض تأخیر است.

راه‌حل: امتیاز ریسک ۱۰۰۰۰ + تحلیل متنی AI.

شاخص	حداکثر	منطق
نسبت تسک‌های عقب‌افتاده	۴۰	عقب‌افتاده ÷ کل باز × 40
نسبت اعضای اضافه‌بار	۳۰	اضافه‌بار ÷ کل × 30
زنجیره‌های پرریسک	۲۰	تعداد × 4 (حداکثر ۲۰)
تمدیدهای اخیر	۱۰	تعداد ۷ روز اخیر × 2 (حداکثر ۱۰)

سطوح: 25-0=کم (سبز)، 50-26=متوسط (زرد)، 75-51=بالا (نارنجی)، 100-76=بحرانی (قرمز)

سرویس: DelayRiskService.cs . تحلیل متنی AI به زبان فارسی با لحن مشاور مدیریت.

نواوری ۸: شاخص سلامت پروژه (Project Health Score)

مشکل: فهمیدن وضعیت کلی پروژه نیازمند ورود به چندین صفحه است.

راه حل: یک عدد ترکیبی ۱۰۰۰۰ با رنگ و نماد.

وزن	زیرشاخص
۳۰٪	سلامت زمان بندی
۲۵٪	سلامت بارکاری
۲۰٪	سلامت وابستگی
۲۵٪	درصد پیشرفت

فرمول: $\text{HealthScore} = \text{Schedule} \times 0.30 + \text{Workload} \times 0.25 + \text{Dependency} \times 0.20 + \text{Delivery} \times 0.25$

سطوح: $85 \leq$ عالی 😊 , $70 \leq$ خوب 😊 , $50 \leq$ نیازمند توجه 😟 , $50 >$ بحرانی 😞

سرویس: ProjectHealthService.cs

نواوری ۹: تولید گزارش هوشمند پایان اسپرینت (AI Sprint Report)

مشکل: نوشتن گزارش پایان اسپرینت کاری زمان بر و تکراری است.

راه حل: AI با تحلیل آمار واقعی، گزارش روایی ۳-۵ جمله ای تولید می کند.

داده های جمع آوری شده: نام اسپرینت، هدف، بازه زمانی، Story Point برنامه ریزی vs تکمیل، تسک های ناتمام، بیشترین مشارکت کننده ها.

سرویس: SprintReportAiService.cs . گزارش ها در SprintReport.cs ذخیره و تاریخچه قابل مشاهده است.

نواوری ۱۰: ترید کردن وظایف بین اعضا (Task Trading)

مشکل: عضو درگیر کار دیگری است و باید تسکش را واگذار کند — بدون اجازه طرف مقابل ممکن نیست.

راه حل: سیستم درخواست و تأییدیه. دو نوع: واگذاری یک طرفه و مبادله دوطرفه.

جزئیات	مقدار
سرویس	TaskTradeService.cs
موجودیت	TaskTradeRequest.cs
وضعیت‌ها	Pending, Accepted, Rejected, Cancelled

قوانین امنیتی: نمی‌توان با خودتان ترید کنید. تسک باید به شما تخصیص داشته باشد. فقط دریافت‌کننده حق پاسخ دارد.

نواوری ۱۱: گراف وابستگی وظایف (Dependency Graph) — در حال پیاده‌سازی

مشکل: خواندن لیست‌های متنی وابستگی‌ها دشوار است.

راه حل: نمای گرافیکی تسک‌ها به صورت گره و یال با رنگ‌بندی وضعیت.

پیاده‌سازی: متد `GetDependencyGraphAsync()` در `TaskDependencyService` . خروجی: `DependencyGraphViewModel1` با `Nodes` و `Edges`.

رنگ‌بندی: سبز=تمام‌شده, قرمز=عقب‌افتاده, نارنجی=پرریسک, خاکستری=عادی

۷. تکنولوژی‌ها و چرایی استفاده

هوش مصنوعی (OpenAI-compatible API)

فایل‌ها: `OpenAiSettings.cs` , `AIClientService.cs` , `IAIClientService.cs`

نحوه اتصال: HTTP POST به `http://92.246.145.99:1234/v1/chat/completions` . مدل Qwen3-4B روی LM Studio .
تایم‌اوت ۶۰ ثانیه، حداکثر ۴۰۰ توکن.

کجاها استفاده شده: (۱) `TaskBreakdownService` — تجزیه تسک, (۲) `DelayRiskService` — تحلیل ریسک, (۳) `SprintReportAiService` — گزارش اسپرینت

ویژگی پرامیت‌ها: همه به فارسی، استفاده از `/no_think` برای جلوگیری از تفکر اضافی، temperature بین ۰.۵ تا ۰.۷.

Email Service

فایل‌ها: `EmailService.cs` , `IServiceEmail` , `EmailSettings.cs`

کجا: `WorkspaceInvitationService` — ارسال ایمیل دعوت‌نامه با لینک ثبت‌نام/پذیرش. قالب HTML با دکمه استایل‌دار.

Webpushr (Push Notification)

فایل: `WebpushrService.cs`

نحوه کار: کاربر اشتراک Push می‌کند → `WebpushrSubscriberId` ذخیره می‌شود → هنگام ارسال پیام چت، به اعضای آفلاین Push ارسال می‌شود.

Background Services

سرویس	وظیفه	فاصله
<code>OverdueCascadeBackgroundService</code>	تمدید خودکار موعد وابسته‌ها	هر ۳۰ دقیقه
<code>ReminderBackgroundService</code>	ارسال یادآوری‌ها	دوره‌ای

۸. فایل ساختار پروژه

```

SmartTask.Web/
├─ Program.cs                # نقطه ورود برنامه
├─ Controllers/              # MVC کنترلر 41
│   ├─ AccountController.cs  # احراز هویت
│   ├─ TaskController.cs     # مدیریت تسک
│   ├─ TaskDependencyController.cs # وابستگی‌ها
│   ├─ OffroadController.cs  # آفرود
│   ├─ WorkloadController.cs # بارکاری
│   ├─ DelayRiskController.cs # ریسک و اولویت هوشمند
│   ├─ TaskTradeController.cs # ترید تسک
│   ├─ SprintReportController.cs # گزارش اسپرینت
│   └─ ... (کنترلر دیگر 32)
├─ Data/
│   ├─ Context/ApplicationDbContext.cs
│   └─ Configurations/      # Fluent API فایل 34
├─ Hubs/
│   ├─ NotificationHub.cs    # (SignalR) هاب اعلان‌ها
│   └─ ChatHub.cs           # (SignalR) هاب چت
├─ Infrastructure/
│   ├─ BackgroundJobs/      # سرویس‌های پس‌زمینه
│   ├─ Repositories/        # GenericRepository + UnitOfWork
│   └─ Seed/                # RoleSeeder + AdminSeeder
├─ Models/
│   ├─ Entities/            # موجودیت 34
│   ├─ Enums/               # شمارشگر 23
│   └─ ViewModels/          # پوشه 30+
├─ Services/
│   ├─ AI/                  # AiClientService
│   ├─ Email/               # EmailService
│   ├─ Interfaces/          # رابط 45
│   └─ Implementations/     # پیاده‌سازی 45
├─ Views/                   # View پوشه 30+
└─ wwwroot/                 # فایل‌های استاتیک

```


۹. تست‌ها و کیفیت کد

تست	سرویس	تمرکز
TaskDependencyServiceTests	TaskDependencyService	چرخه وابستگی، BFS، محاسبه تأخیر
WorkloadAnalysisServiceTests	WorkloadAnalysisService	محاسبه بارکاری
PriorityEngineServiceTests	PriorityEngineService	امتیازدهی هوشمند
ProjectHealthServiceTests	ProjectHealthService	شاخص سلامت
SprintServiceTests	SprintService	مدیریت اسپرینت
SubTaskServiceTests	SubTaskService	زیروظایف
TaskServiceTests	TaskService	CRUD تسک
DateFormatServiceTests	DateFormatService	تاریخ شمسی/میلادی

نکات بهینه‌سازی

- **Soft Delete** با ViewState در BaseEntity
- **ExecuteUpdateAsync** به جای Load-Modify-Save
- **Batch Operations** برای اعلان‌ها و فعالیت‌ها
- **In-Memory Traversal** برای گراف وابستگی (جلوگیری از N+1 Query)
- **Pre-computed Maps** برای محاسبه بارکاری

۱۰. پرسش‌های متداول اساتید

س: چرا از هوش مصنوعی استفاده کردید؟

ج: در سه بخش: (۱) تجزیه خودکار تسک‌ها برای اعضای کم‌تجربه، (۲) تحلیل متنی ریسک پروژه به جای اعداد خام، (۳) تولید خودکار گزارش اسپرینت. هدف: کاهش بار کاری مدیر و افزایش سرعت تصمیم‌گیری.

س: تفاوت SmartTask با Jira یا Trello چیست؟

ج: (۱) بخش آفورد، (۲) تحلیل بارکاری خودکار، (۳) تمدید خودکار وابسته‌ها، (۴) اولویت‌بندی هوشمند، (۵) گزارش‌گیری AI، تجزیه تسک با (۷) AI، ترید تسک با سیستم درخواست و تأییدیه.

س: چرا SignalR استفاده کردید؟

ج: (۱) اعلان‌های فوری — تخصیص تسک جدید → اعلان فوراً برسد، (۲) چت گروهی — پیام‌رسانی Real-time بدون رفرش صفحه.

س: مکانیزم جلوگیری از حلقوی شدن وابستگی‌ها چیست؟

ج: الگوریتم BFS معکوس. وقتی کاربر B را وابسته به A می‌کند، سیستم از B شروع به پیمایش می‌کند. اگر به A برسد → حلقوی است → خطا.

س: تمدید خودکار چگونه از تکرار جلوگیری می‌کند؟

ج: جدول OverdueCascadeLog ثبت می‌کند چند روز اعمال شده. اگر تأخیر جدید بیشتر باشد → فقط مابه‌التفاوت. اگر کمتر/مساوی → هیچ کاری.

س: هوش مصنوعی چگونه متصل می‌شود؟

ج: HTTP POST به API سازگار با OpenAI. مدل Qwen3-4B روی LM Studio. تایم‌اوت ۶۰ ثانیه، حداکثر ۴۰۰ توکن. پرامپت‌ها به فارسی.